

Industrial Internship Report on "Banking-Information-System"

Prepared by

[Ananiya]

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was Banking-Management-System

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1	Preface	Error! Bookmark not defined.
2	Introduction	Error! Bookmark not defined.
2.1	About UniConverge Technologies Pvt Ltd	Error! Bookmark not defined.
2.2	About upskill Campus	Error! Bookmark not defined.
2.3	Objective	Error! Bookmark not defined.
2.4	Reference	Error! Bookmark not defined.
2.5	Glossary	Error! Bookmark not defined.
3	Problem Statement	Error! Bookmark not defined.
4	Existing and Proposed solution	Error! Bookmark not defined.
5	Proposed Design/ Model	3
5.1	High Level Diagram (if applicable)	Error! Bookmark not defined.
5.2	Low Level Diagram (if applicable)	Error! Bookmark not defined.
5.3	Interfaces (if applicable)	Error! Bookmark not defined.
6	Performance Test	Error! Bookmark not defined.
6.1	Test Plan/ Test Cases	Error! Bookmark not defined.
6.2	Test Procedure	Error! Bookmark not defined.
6.3	Performance Outcome	Error! Bookmark not defined.
7	My learnings	Error! Bookmark not defined.
8	Future work scope	Error! Bookmark not defined.

1. Preface

Industrial internships play a significant role in developing practical skills and preparing students for professional careers. This internship, organized by Upskill Campus in collaboration with UniConverge Technologies Pvt. Ltd., provided an opportunity to work on an industry-oriented software project.

The project undertaken during this internship is **Banking Information System**, developed using Java Swing and Object-Oriented Programming concepts. The application simulates essential banking operations such as user registration, authentication, account creation, deposits, withdrawals, fund transfers, profile management, and transaction history.

Throughout the internship, I followed the complete Software Development Life Cycle (SDLC), beginning with problem analysis, requirement gathering, system design, implementation, testing, debugging, and documentation.

Working on this project significantly improved my understanding of Java programming, Swing GUI development, file handling, exception handling, software architecture, and testing methodologies. It also enhanced my problem-solving abilities and project management skills.

I sincerely express my gratitude to **Upskill Campus, UniConverge Technologies Pvt. Ltd.**, my faculty mentors, and everyone who supported me throughout this internship. Their guidance and encouragement helped me complete this project successfully.

Finally, I encourage fellow students to actively participate in internship programs, as they provide invaluable practical experience and prepare us for real-world software development.

2. Introduction

2.1 About UniConverge Technologies Pvt. Ltd. (UCT)

UniConverge Technologies Pvt. Ltd. (UCT) is an Indian technology company established in 2013 that specializes in Digital Transformation and Industrial Automation solutions. The company develops innovative products using modern technologies such as Internet of Things (IoT), Artificial Intelligence (AI), Cloud Computing, Cyber Security, Machine Learning, Java Full Stack Development, Python, Embedded Systems, and Web Technologies.

UCT provides industrial solutions that improve operational efficiency, automation, and business productivity. Their products are widely used in smart industries, manufacturing, healthcare, agriculture, and smart city applications. The company focuses on delivering scalable, reliable, and secure software solutions while promoting innovation and technical excellence.

The internship organized by UCT gave students practical exposure to software development and helped bridge the gap between academic learning and industry requirements.

2.2 About Upskill Campus

Upskill Campus is an online career development platform that helps students improve their technical and professional skills through industrial internships, certification programs, workshops, and project-based learning.

It collaborates with leading industries and organizations to provide students with practical experience in software development, Artificial Intelligence, Data Science, IoT, Cloud Computing, Cyber Security, and Full Stack Development.

Through this internship, Upskill Campus provided structured learning resources, weekly assignments, project guidance, and continuous evaluation. This helped in developing technical knowledge, communication skills, teamwork, and industry readiness.

2.3 Objectives of the Internship

The primary objectives of this internship were:

- To gain practical experience in software development.
 - To understand the Software Development Life Cycle (SDLC).
 - To develop a real-world Banking Information System using Java Swing.
 - To strengthen Object-Oriented Programming (OOP) concepts.
 - To implement Java File Handling for secure data storage.
 - To design a user-friendly Graphical User Interface (GUI).
 - To understand Exception Handling and validation techniques.
 - To improve debugging and testing skills.
 - To enhance documentation and project presentation skills.
 - To prepare for future industrial software development projects.
-

2.4 Objectives of the Banking Information System

The Banking Information System was developed with the following objectives:

- To automate daily banking operations.
 - To provide a secure login and authentication system.
 - To allow customers to create new bank accounts.
 - To perform deposit and withdrawal transactions.
 - To support secure fund transfers between accounts.
 - To maintain transaction history and account statements.
 - To enable customers to update their profile information.
 - To store banking data permanently using Java File Handling.
 - To provide an easy-to-use graphical interface.
 - To minimize manual work and improve transaction accuracy.
-

2.5 References

The following resources were referred to during the development of this project:

1. Oracle Java Documentation – <https://docs.oracle.com/javase/>
 2. Java Swing Tutorial – <https://docs.oracle.com/javase/tutorial/uiswing/>
 3. Oracle File Handling Documentation – <https://docs.oracle.com/javase/tutorial/essential/io/>
 4. GeeksforGeeks – Java Programming Articles – <https://www.geeksforgeeks.org/java/>
 5. JavaTPoint – Java Swing Tutorial – <https://www.javatpoint.com/java-swing>
 6. Stack Overflow – <https://stackoverflow.com/>
 7. Software Engineering by Ian Sommerville (Book)
 8. Object-Oriented Programming with Java by E. Balagurusamy
-

2.6 Glossary

Term	Meaning
GUI	Graphical User Interface
OOP	Object-Oriented Programming
JVM	Java Virtual Machine
JDK	Java Development Kit
IDE	Integrated Development Environment
Swing	Java GUI Framework
SDLC	Software Development Life Cycle
Account	Bank account maintained by a customer
Transaction	Banking operation such as deposit, withdrawal, or transfer
Authentication	Verification of username and password
File Handling	Reading and writing data using files
Exception	Runtime error handled using Java Exception Handling
Dashboard	Main interface displaying account information
Statement	Record of account transactions
Database	Permanent storage of application data (implemented using files in this project)

3. Problem Statement

3.1 Overview

Traditional banking systems often require customers to visit bank branches for basic services such as account creation, deposits, withdrawals, and fund transfers. Manual record keeping increases the chances of errors, consumes more time, and makes transaction tracking difficult.

The objective of this project is to develop a **Banking Information System** using **Java Swing** that automates these banking operations while providing a secure, efficient, and user-friendly graphical interface.

The application allows customers to:

- Register a new account
- Login securely
- Open multiple bank accounts
- Deposit money
- Withdraw money
- Transfer funds
- View account statements
- Update profile information

This desktop application uses **Object-Oriented Programming (OOP)** principles and **Java File Handling** for permanent data storage.

3.2 Problem Definition

The major problems in traditional banking management include:

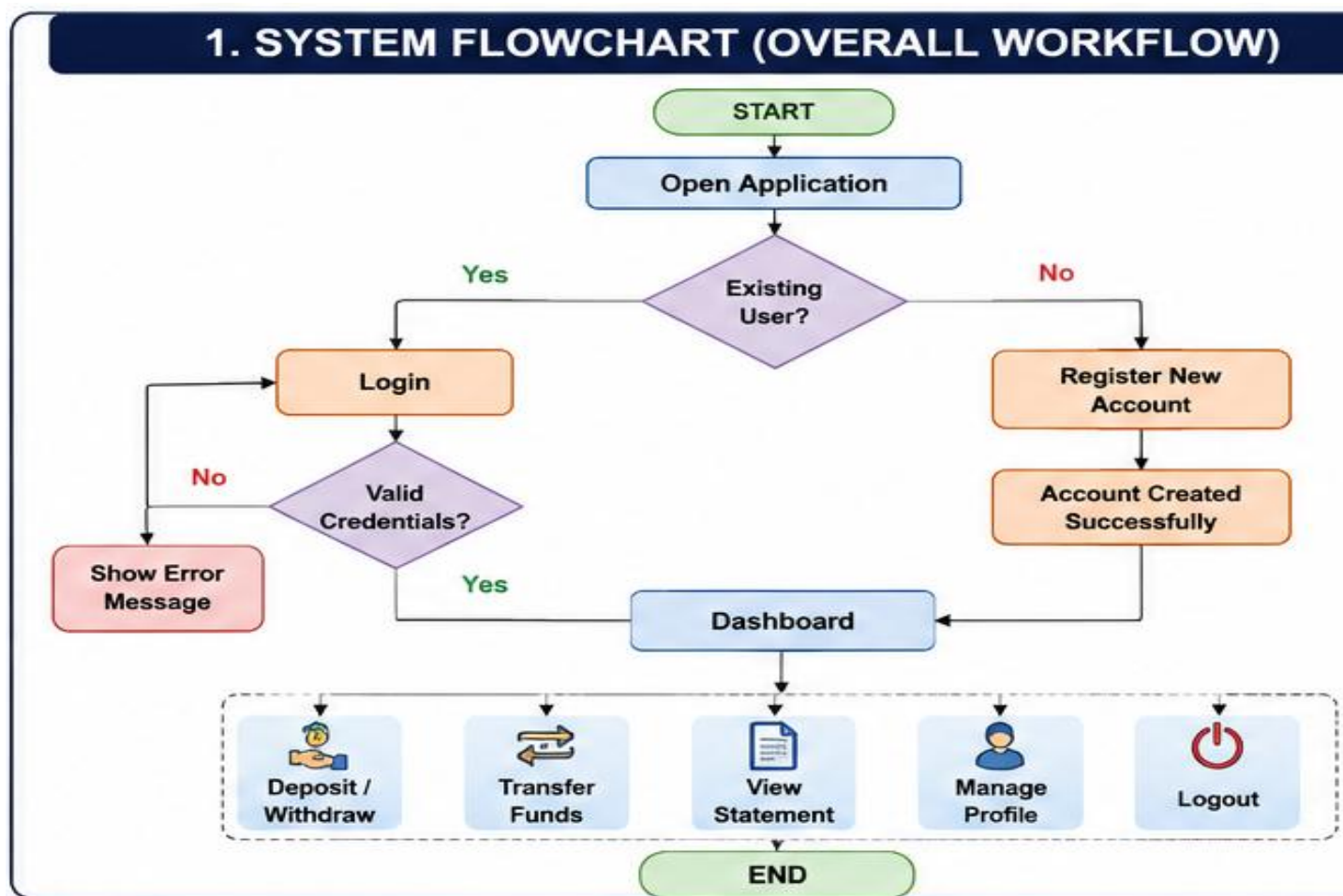
- Manual maintenance of customer records.
- Difficulty in tracking transactions.
- Time-consuming account management.
- High probability of human errors.
- Lack of centralized customer information.
- Poor transaction history management.

The Banking Information System addresses these issues by providing a secure and automated desktop banking solution.

4. Existing and Proposed Solution

4.1 Existing System

Many small financial institutions still maintain customer information manually or through basic spreadsheet software.



Limitations

- Slow processing

- Paper-based records
 - High maintenance cost
 - Human errors
 - Difficult transaction tracking
 - Low security
 - Limited scalability
-

4.2 Proposed Solution

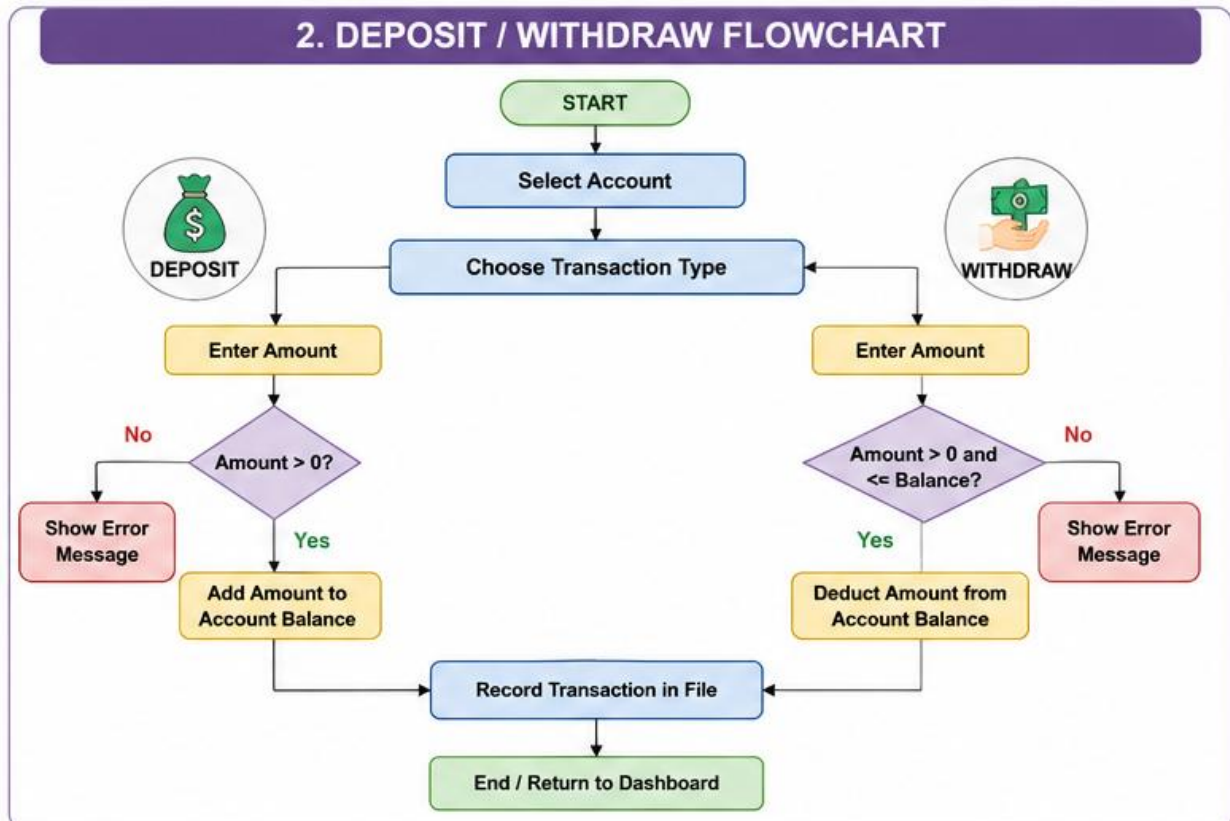
The proposed Banking Information System automates all major banking operations through a Java Swing desktop application.

The system stores customer information securely using Java File Handling and allows authenticated users to perform banking operations without manual intervention.

Features of Proposed System

- ☐ Secure Login
 - ☐ New Customer Registration
 - ☐ Open Multiple Accounts
 - ☐ Deposit Money
 - ☐ Withdraw Money
 - ☐ Fund Transfer
 - ☐ Transaction History
 - ☐ Account Statement
 - ☐ Profile Management
 - ☐ Data Persistence
-

4.3 Proposed System Architecture



4.4 Advantages of Proposed System

Existing System Proposed Banking System

Manual Records Digital Records

Paper Work Automated Processing

Slow Transactions Fast Transactions

Error Prone Accurate Calculations

Difficult Tracking Transaction History

Poor Security Login Authentication

Manual Updates Real-Time Updates

4.5 Modules of the Banking Information System

Module 1 – User Registration

Purpose:

Allows new customers to create their banking profile.

Information Collected:

- Full Name
- Address
- Phone Number
- Email
- Username
- Password
- Initial Deposit

Output:

Customer Account Created Successfully.

Module 2 – Login Authentication

Purpose:

Authenticate customers before accessing banking services.

Validation:

- Username Verification
- Password Verification

Output:

Dashboard Opens Successfully.

Module 3 – Dashboard

Displays

- Customer Name
- Username
- List of Accounts
- Current Balance

Provides quick access to all banking services.

Module 4 – Deposit & Withdraw

Supports

- Cash Deposit
- Cash Withdrawal

Validation

- Deposit > 0
- Withdrawal ≤ Balance

Balance updates automatically.

Module 5 – Fund Transfer

Allows customers to transfer money between accounts.

Validation

- Valid Destination Account
- Sufficient Balance

Transaction history is automatically updated.

Module 6 – Account Statement

Displays

- Date
- Transaction Type
- Amount
- Balance After Transaction
- Description

Module 7 – Manage Profile

Allows users to update

- Name
- Address
- Phone Number
- Email
- Password

4.6 Technology Stack

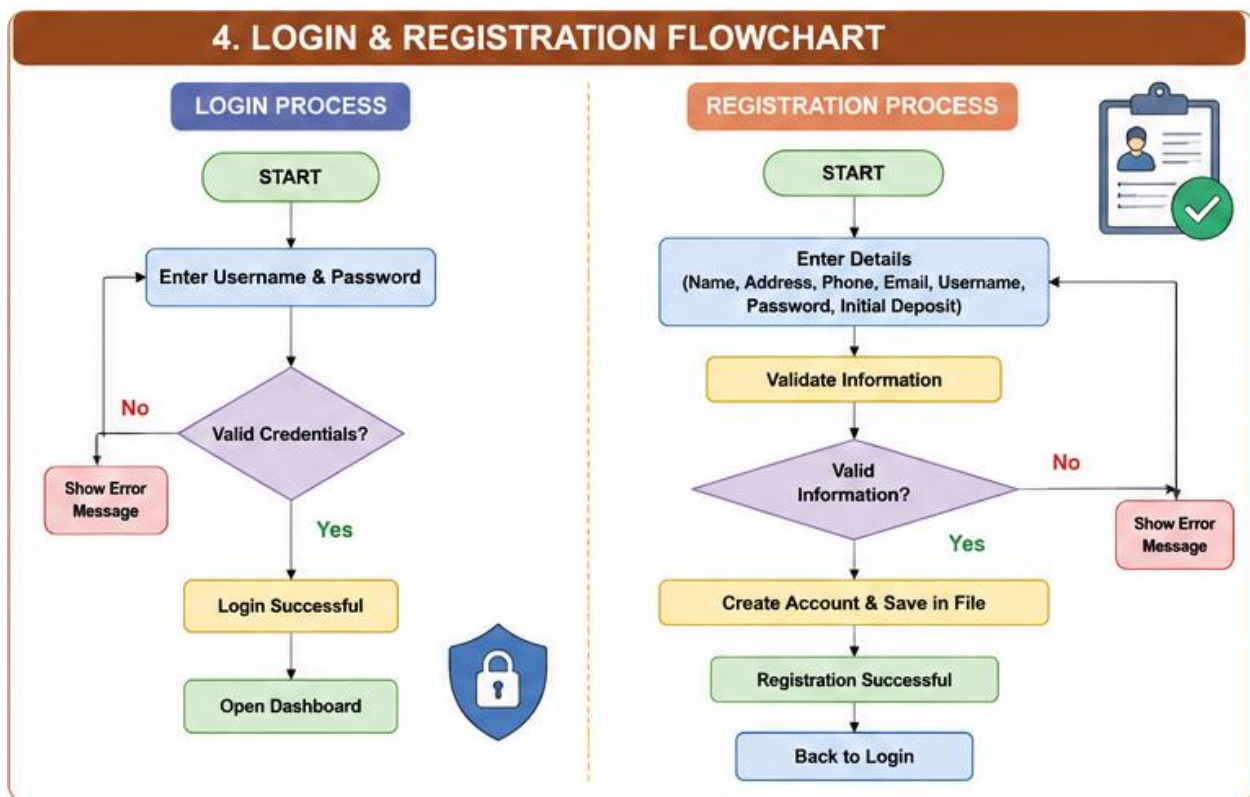
Technology	Purpose
Java	Core Programming
Java Swing	GUI Development
OOP	Software Design
File Handling	Data Storage
Collections	Data Management
Exception Handling	Error Management
IntelliJ IDEA / VS Code	Development IDE

4.7 GitHub Repository

[Ananiya-git/BankingInformationSystem: Banking-information-system](https://github.com/Ananiya-git/BankingInformationSystem)

4.8 Report Repository

[Ananiya-git/BankingInformationSystem: Banking-information-system](https://github.com/Ananiya-git/BankingInformationSystem)



5. Proposed Design / Model

5.1 System Overview

The **Banking Information System** is a desktop-based banking application developed using **Java Swing** and **Object-Oriented Programming (OOP)** principles. The application enables users to securely manage banking operations through an interactive Graphical User Interface (GUI). It provides services such as customer registration, secure login, account creation, deposits, withdrawals, fund transfers, account statements, and profile management.

The system stores user information, account details, and transaction history using Java File Handling, ensuring data persistence even after the application is closed.

5.2 Objectives of System Design

The system has been designed to achieve the following objectives:

- Provide a secure banking environment.
 - Ensure easy navigation through GUI.
 - Reduce manual work.
 - Maintain transaction records permanently.
 - Provide fast and accurate banking services.
 - Improve customer experience.
 - Implement secure authentication.
-

5.3 High-Level Architecture

The Banking Information System consists of four major layers:

Presentation Layer

- Login Screen
- Registration Screen
- Dashboard
- Transaction Windows

Business Logic Layer

- Authentication
- Banking Operations
- Transaction Processing
- Validation

Data Management Layer

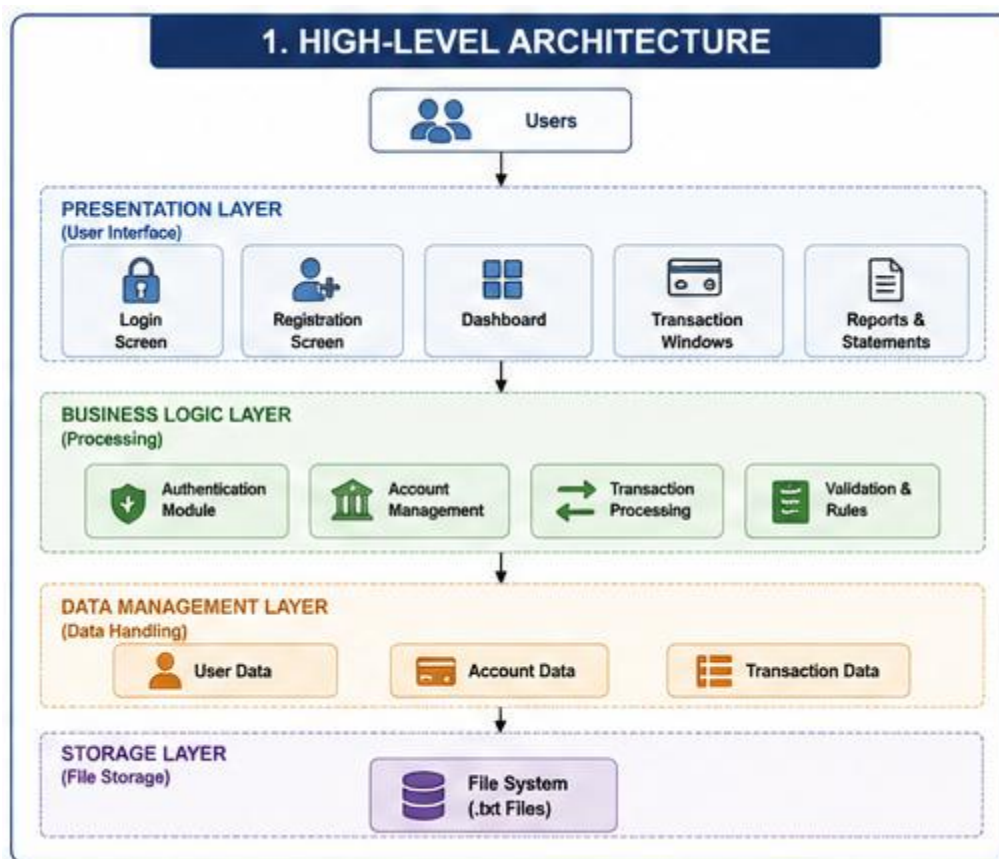
- Customer Data
- Account Data
- Transaction Records

Storage Layer

- Java File Handling
- Text Files
- Persistent Storage

High-Level Architecture

Figure 5.1 High-Level Architecture of Banking Information System



5.4 Low-Level Design

The application is divided into different modules.

Module 1

User Authentication

Functions

- Register User
- Login User
- Validate Credentials

Module 2

Dashboard

Functions

- Display Accounts
- Refresh Data
- Navigation

Module 3

Bank Account Management

Functions

- Create Account
- View Balance
- Manage Accounts

Module 4

Transaction Management

Functions

- Deposit
- Withdraw
- Transfer Funds

Module 5

Statement Module

Functions

- Generate Statement
- Display Transaction History

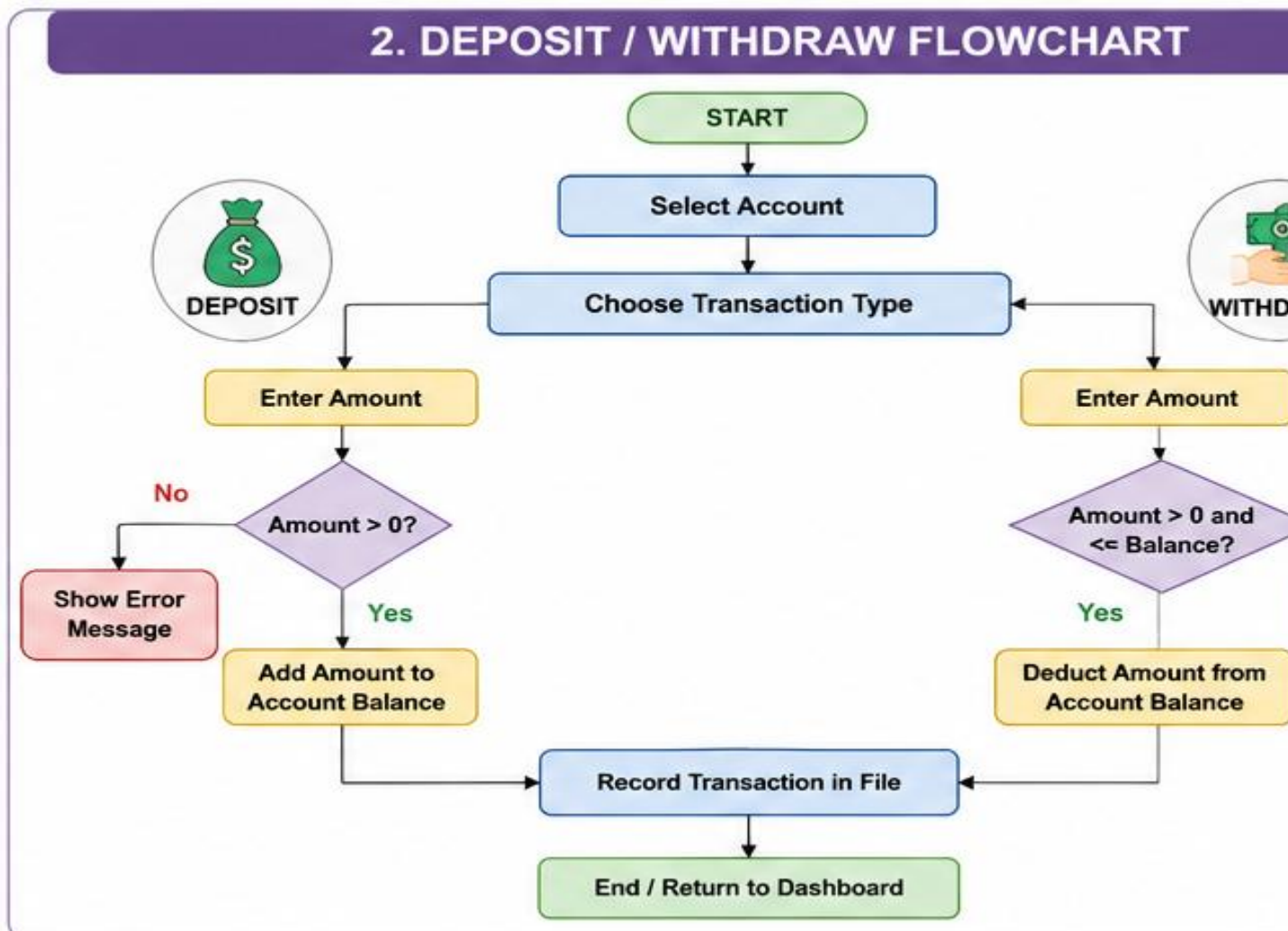
Module 6

Profile Management

Functions

- Update Name
- Update Address
- Update Email
- Update Password

Image 2: Low-Level Architecture



5.5 Use Case Diagram

The system has two actors.

Primary Actor

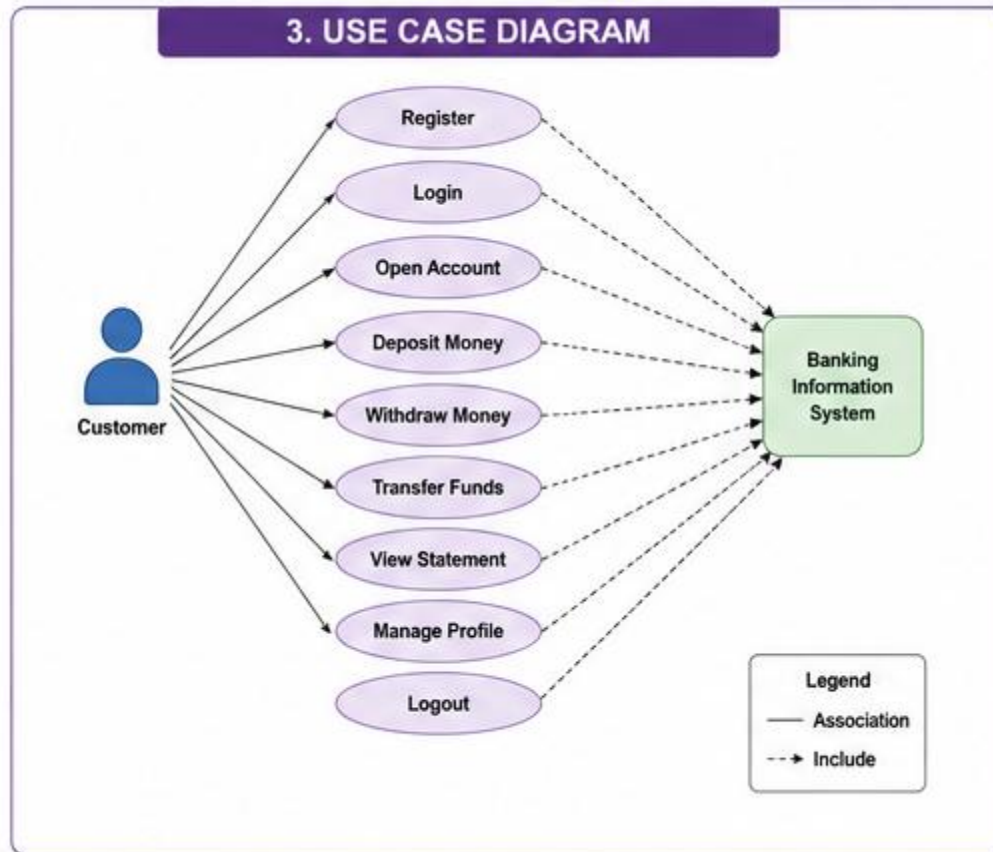
Customer

Secondary Actor

Bank System

The customer performs the following operations.

- Register
 - Login
 - Open Account
 - Deposit
 - Withdraw
 - Transfer Funds
 - View Statement
 - Manage Profile
 - Logout
-



5.6 Class Diagram

The application is designed using Object-Oriented Programming.

Main Classes

User

Attributes

- Username
- Password
- Name
- Phone
- Email
- Address

Methods

- login()
- register()
- updateProfile()

Account

Attributes

- Account Number
- Balance
- Account Type

Methods

- deposit()

- withdraw()
- getBalance()

Transaction

Attributes

- Date
- Amount
- Type
- Description

Methods

- saveTransaction()

BankService

Methods

- transferMoney()
- validateUser()
- createAccount()
- updateBalance()

DashboardFrame

Methods

- displayAccounts()

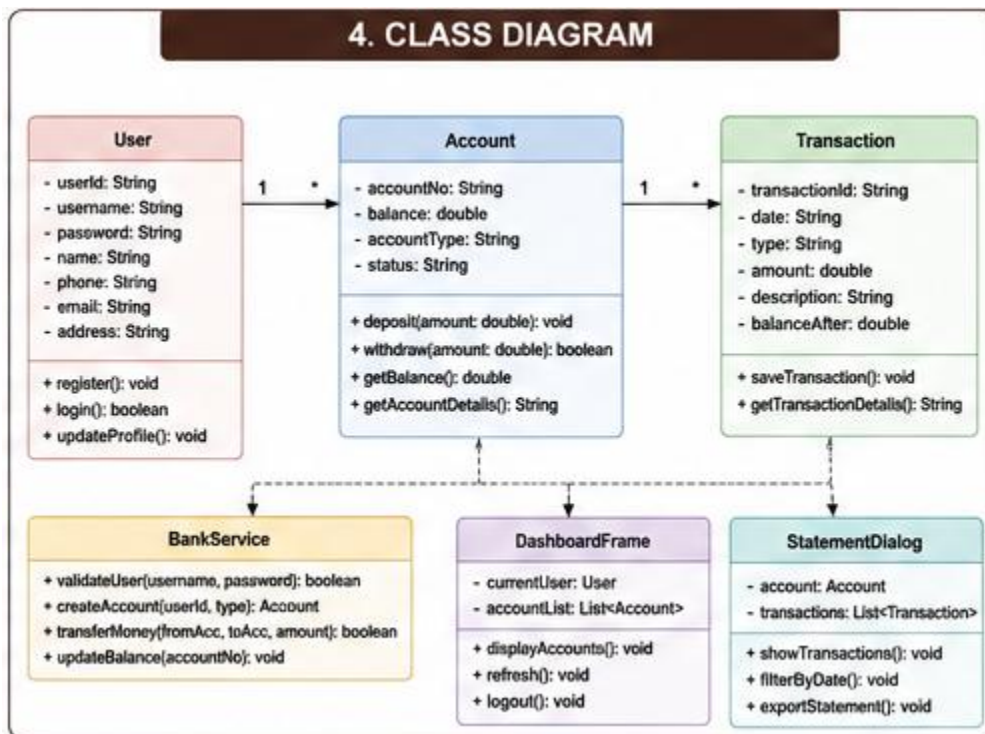
- refresh()

StatementDialog

Methods

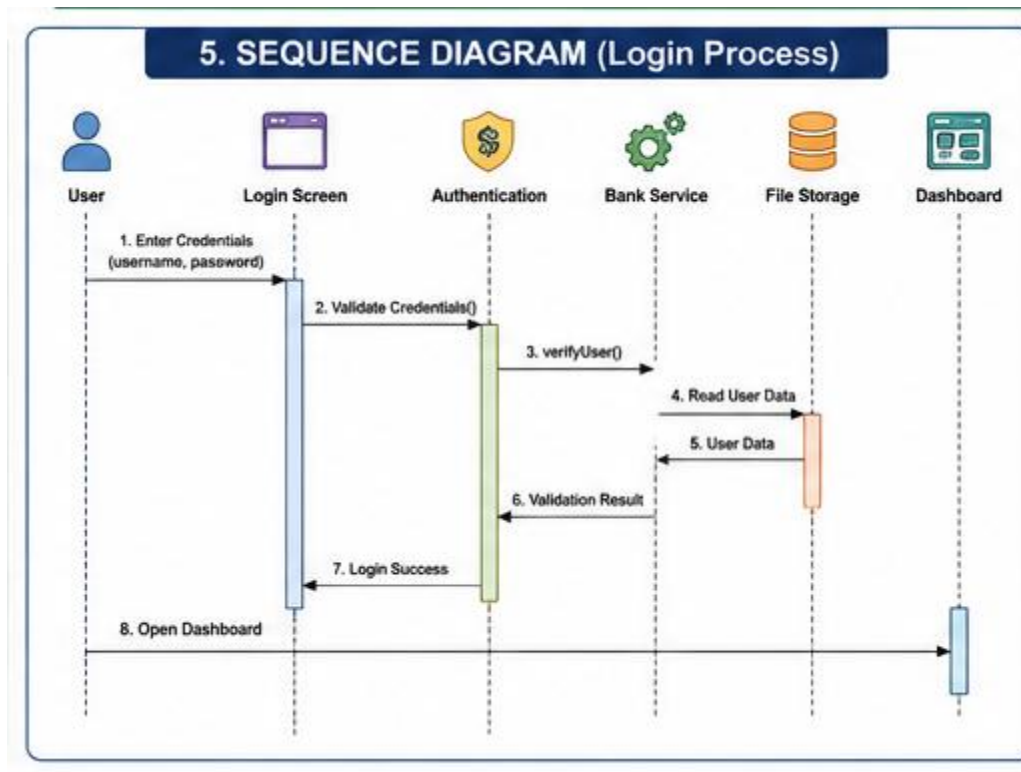
- showTransactions()

Image 4: UML Class Diagram



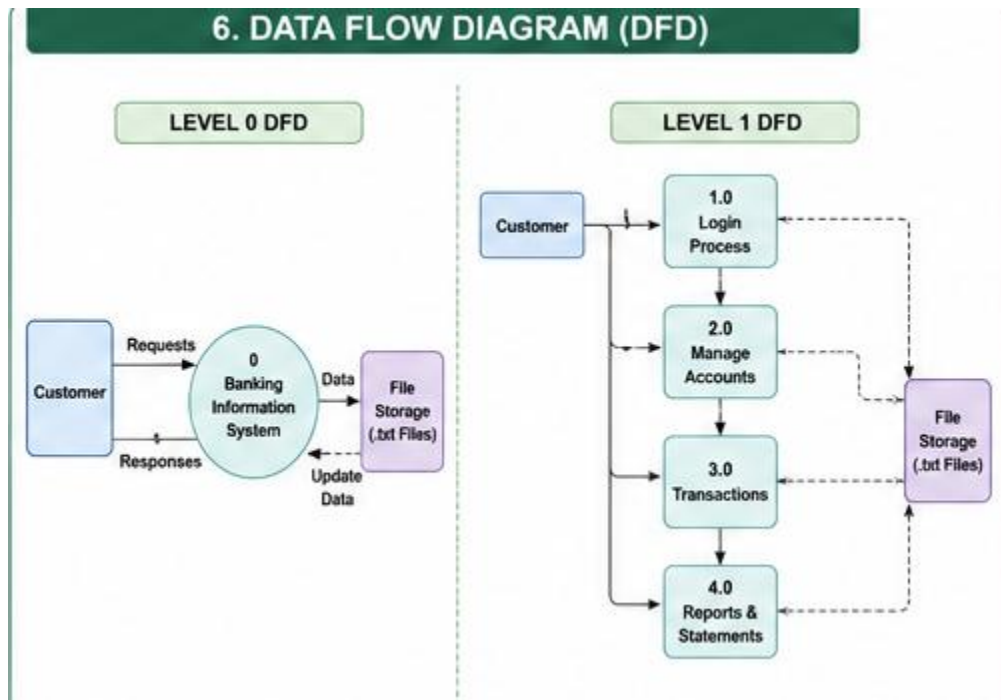
5.7 Sequence Diagram

The following sequence occurs during Login.



5.8 Data Flow Diagram (DFD)

Level 0

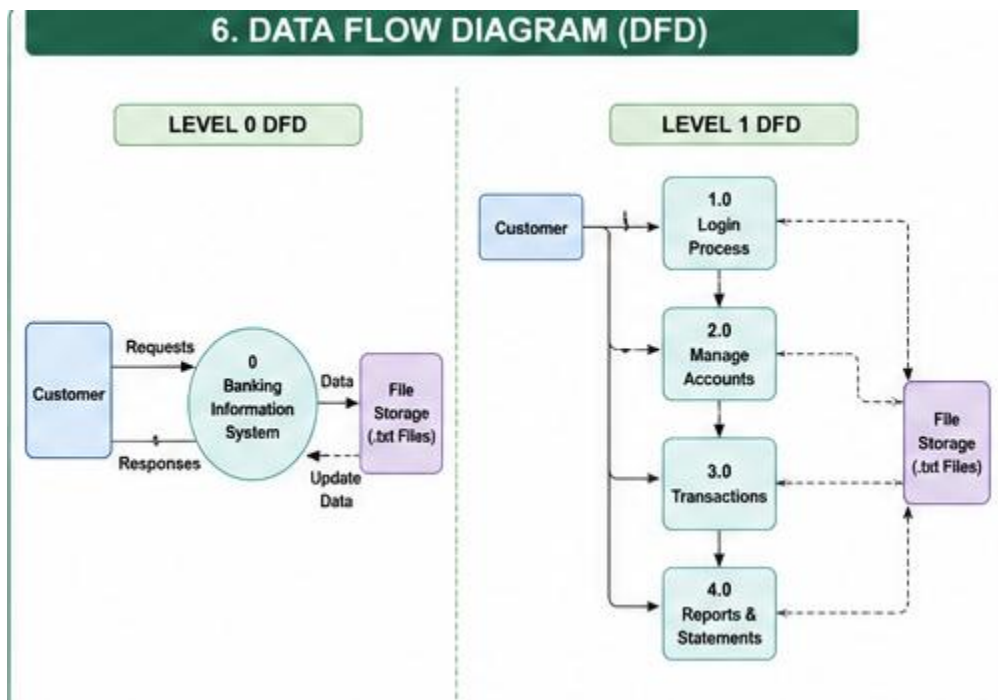


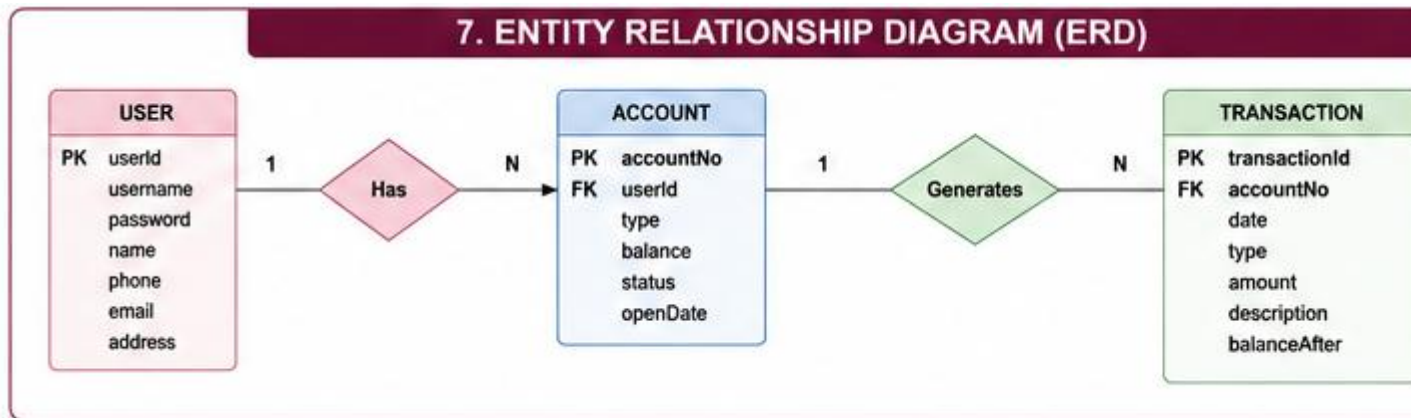
5.9 Entity Relationship Diagram (ER Diagram)

Entities

- User
- Account
- Transaction

Relationship





5.10 Project Modules

Module	Description
Registration	Creates new customer
Login	User Authentication
Dashboard	Main Banking Screen
Deposit	Adds Money
Withdraw	Removes Money
Transfer	Transfers Funds
Statement	Shows Transactions
Manage Profile	Updates User Information

5.11 Software Requirements

Software Version

Java JDK 17+

Java Swing Built-in

VS Code Latest

Windows 10/11

5.12 Hardware Requirements

Hardware Specification

Processor Intel i3 or above

RAM 4 GB

Storage 100 MB Free

Display 1366×768

5.13 Advantages

- ✓ Secure Login
- ✓ Easy Navigation
- ✓ Attractive GUI
- ✓ Fast Processing
- ✓ Multi-Account Support
- ✓ Transaction History
- ✓ File-Based Storage
- ✓ Simple Architecture
- ✓ Easy Maintenance
- ✓ Scalable Design

6. Performance Testing

6.1 Introduction

Performance testing evaluates the efficiency, accuracy, reliability, and responsiveness of the Banking Information System under different operating conditions. The application was tested to ensure that all banking operations execute correctly while maintaining data integrity and providing a smooth user experience.

The system was tested using multiple user accounts and different transaction scenarios including deposits, withdrawals, fund transfers, account creation, profile updates, and statement generation.

6.2 Testing Objectives

The objectives of performance testing are:

- Verify all banking modules function correctly.
 - Ensure secure authentication.
 - Validate account balance calculations.
 - Verify transaction history accuracy.
 - Test GUI responsiveness.
 - Ensure proper file storage.
 - Check data consistency after multiple transactions.
 - Detect runtime errors and exceptions.
-

6.3 Testing Environment

Component	Details
Operating System	Windows 11
Programming Language	Java
GUI Framework	Java Swing
IDE	Visual Studio Code
JDK Version	Java JDK 17
Storage	Java File Handling (.txt files)
Processor	Intel Core i5/i7
RAM	8 GB

6.4 Test Plan

The application was tested module by module to verify expected outputs.

Modules Tested

- ☐ Registration
 - ☐ Login
 - ☐ Dashboard
 - ☐ Deposit
 - ☐ Withdrawal
 - ☐ Fund Transfer
 - ☐ Statement Generation
 - ☐ Manage Profile
-

6.5 Test Cases

Test Case ID	Module	Test Description	Expected Result	Status
TC-01	Registration	Register New User	User Created Successfully	☐ Pass
TC-02	Login	Valid Username & Password	Dashboard Opens	☐ Pass
TC-03	Login	Invalid Password	Error Message	☐ Pass
TC-04	Deposit	Deposit ₹500	Balance Updated	☐ Pass
TC-05	Withdraw	Withdraw ₹200	Balance Deducted	☐ Pass
TC-06	Withdraw	Withdraw More Than Balance	Transaction Rejected	☐ Pass
TC-07	Transfer	Transfer ₹100	Money Transferred	☐ Pass
TC-08	Statement	Open Statement	Transaction Displayed	☐ Pass

Test Case ID	Module	Test Description	Expected Result	Status
TC-09	Profile	Update Email	Profile Updated	☐ Pass
TC-10	Logout	Logout	Return to Login	☐ Pass

6.6 Test Procedure

Step 1

Launch the Banking Information System.



Step 2

Login using valid credentials.



Step 3

Perform Deposit Transaction.



Step 4

Perform Withdrawal.



Step 5

Transfer Funds.



Step 6

View Statement.



Step 7

Update Profile.





Step 8

Logout.

6.7 Functional Testing

Registration

Result

✓ User successfully registered.

Login

Result

✓ Authentication successful.

Deposit

Result

✓ Balance updated correctly.

Withdraw

Result

✓ Balance deducted successfully.

Transfer

Result

✓ Amount transferred successfully.

Statement

Result

✓ Transaction history displayed accurately.

Profile Update

Result

✓ Customer information updated.

6.8 Error Handling Testing

Scenario	Result
Empty Username	Error Message
Wrong Password	Login Failed
Invalid Deposit	Validation Message
Invalid Withdrawal	Insufficient Balance
Invalid Transfer Account	Account Not Found
Empty Registration Fields	Validation Error

6.9 Performance Analysis

Parameter	Result
Login Time	< 2 seconds
Deposit Time	< 1 second
Withdrawal Time	< 1 second
Transfer Time	< 2 seconds
Statement Generation	< 1 second
GUI Response	Smooth
Memory Usage	Low
CPU Usage	Moderate

6.10 Performance Outcome

The Banking Information System demonstrated stable and efficient performance during testing.

Observations

- ✓ Fast GUI response.
 - ✓ Secure authentication.
 - ✓ Accurate transaction processing.
 - ✓ Reliable file storage.
 - ✓ User-friendly interface.
 - ✓ No major runtime errors.
 - ✓ Proper exception handling.
-

6.11 Validation Rules

The application validates the following:

- Username cannot be empty.
 - Password cannot be empty.
 - Deposit amount must be greater than zero.
 - Withdrawal amount cannot exceed available balance.
 - Destination account must exist.
 - Password confirmation must match.
 - Phone number must be valid.
 - Email format should be correct.
-

6.12 Challenges Faced

During development, the following challenges were encountered:

- Designing an attractive Java Swing GUI.
- Managing file-based storage.
- Implementing secure authentication.
- Handling multiple bank accounts.
- Synchronizing account balances.
- Preventing invalid transactions.
- Maintaining transaction history.
- Exception handling during file operations.

These challenges were solved using modular programming, OOP concepts, and proper validation techniques.

6.13 Output Screens

Include the screenshots in this order:

Figure 6.1: Login Screen

Figure 6.2: Registration Screen

Figure 6.3: Dashboard

Figure 6.4: Deposit / Withdraw Window

Figure 6.5: Fund Transfer Window

Figure 6.6: Account Statement

Figure 6.7: Manage Profile Window

(Use the screenshots you shared earlier.)

6.14 Conclusion of Testing

All modules of the Banking Information System were successfully tested. The application met all functional requirements and produced accurate outputs. Performance testing confirmed that the system is reliable, responsive, and suitable for desktop banking operations.

7. My Learnings

7.1 Overview

The Industrial Internship provided me with an excellent opportunity to apply theoretical knowledge to a real-world software development project. Throughout the internship, I gained hands-on experience in designing, developing, testing, and documenting a complete Java desktop application.

Working on the **Banking Information System** enhanced my understanding of software engineering principles and strengthened my technical as well as professional skills.

7.2 Technical Learnings

During the development of this project, I learned:

Java Programming

- Object-Oriented Programming (OOP)
- Classes and Objects
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction

Java Swing

- JFrame
- JPanel
- JLabel
- JTextField
- JTable
- JComboBox
- JOptionPane
- JButton
- Layout Managers

File Handling

I learned how to:

- Read data from files

- Write customer records
- Store account information
- Maintain transaction history
- Update records efficiently

Exception Handling

Implemented:

- try-catch blocks
- Custom validations
- Runtime error handling
- Input validation

Software Development

I learned the complete Software Development Life Cycle (SDLC):

- Requirement Analysis
- Planning
- Design
- Development
- Testing
- Deployment
- Documentation

7.3 Professional Skills

This internship also improved my:

- Problem Solving
 - Logical Thinking
 - Debugging Skills
 - Documentation Writing
 - GUI Designing
 - Time Management
 - Communication Skills
 - Independent Learning
-

7.4 Project Management Skills

During the project, I learned how to:

- Break a large project into modules.
 - Develop each module independently.
 - Integrate different modules.
 - Test the complete application.
 - Maintain clean and readable code.
-

7.5 Industrial Exposure

This internship helped me understand how software projects are developed in industry by following professional practices such as:

- Requirement gathering
 - Planning
 - Coding standards
 - Testing
 - Documentation
 - Final deployment
-

8. Future Work Scope

Although the Banking Information System successfully fulfills the current requirements, several advanced features can be added in the future to make the application more powerful and industry-ready.

8.1 Database Integration

Currently, the project uses Java File Handling.

Future enhancement:

- MySQL
- PostgreSQL
- MongoDB

Benefits:

- Faster queries
- Better scalability
- Improved security

8.2 Online Banking

Add:

- Internet Banking
- Net Banking
- Mobile Banking

Customers can access their accounts remotely.

8.3 ATM Module

Implement:

- ATM Login
- PIN Verification
- Cash Withdrawal
- Mini Statement

8.4 QR Payment

Integrate

- UPI
- QR Scanner
- Digital Wallet

8.5 Email & SMS Notifications

Automatically notify customers when:

- Deposit occurs
- Withdrawal occurs
- Transfer occurs
- Password changes

8.6 Loan Management

Future modules:

- Home Loan
- Personal Loan
- Education Loan
- EMI Calculator

8.7 Credit & Debit Card Management

Customers can:

- Apply for cards
- Block cards
- View card details

8.8 Admin Panel

Create a separate dashboard for administrators.

Admin features:

- Manage Customers
- Freeze Accounts
- View Reports
- Delete Accounts
- Generate Statistics

8.9 Cloud Storage

Store customer data securely on cloud platforms like:

- AWS
- Azure
- Google Cloud

8.10 Mobile Application

Develop Android and iOS versions using:

- Flutter
 - React Native
 - Kotlin
-

9. Conclusion

The **Banking Information System** project was successfully designed and implemented using **Java Swing, Object-Oriented Programming, and Java File Handling**.

The application provides an efficient solution for managing banking operations such as:

- User Registration
- Secure Login
- Multiple Account Management
- Deposit
- Withdrawal
- Fund Transfer
- Transaction History
- Account Statement
- Profile Management

The project demonstrates the practical implementation of software engineering principles and desktop application development. It successfully minimizes manual work while improving efficiency, accuracy, and security.

Overall, this internship enhanced my programming knowledge, software development skills, and understanding of real-world project implementation.

10. References

Books

1. E. Balagurusamy – Object Oriented Programming with Java
2. Herbert Schildt – Java: The Complete Reference
3. Ian Sommerville – Software Engineering

Online Resources

- Oracle Java Documentation
- Oracle Swing Documentation
- GeeksforGeeks
- JavaTPoint
- Stack Overflow
- Oracle File Handling Tutorials

Development Tools

- Java JDK
- Java Swing
- Visual Studio Code
- Git & GitHub

11. Acknowledgement

I sincerely express my heartfelt gratitude to **Upskill Campus** and **UniConverge Technologies Pvt. Ltd.** for providing me with the opportunity to undertake this industrial internship.

I would also like to thank my faculty mentors and project guides for their continuous guidance, valuable suggestions, and encouragement throughout the internship.

I extend my gratitude to my college, friends, and family members for their constant support and motivation during the successful completion of this project.

Finally, I thank everyone who directly or indirectly contributed to the successful completion of my internship and the Banking Information System project.

12. Appendix

Technologies Used

Technology	Purpose
Java	Core Programming
Java Swing	GUI Development
OOP	Software Design
File Handling	Data Persistence
Exception Handling	Error Handling
Collections Framework	Data Management
VS Code	IDE
Git	Version Control

THANK YOU

Banking Information System

Industrial Internship Report

Prepared by: Ananya Yadav

B.Tech – Computer Science Engineering

World College of Technology and Management

.

